

Practical (OSSP)

SESSION 5

TASK 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficient.

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2 producer
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>

#define DATA_SIZE 100000

int main() {
    int pipefd[2];
    pid_t pid;
    int data[DATA_SIZE];

    struct timeval start, end;

    // Create pipe
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Generate data
    for (int i = 0; i < DATA_SIZE; i++) {
        data[i] = i + 1;
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2 producer_consumer
gettimeofday(&end, NULL);

long seconds = end.tv_sec - start.tv_sec;
long microseconds = end.tv_usec - start.tv_usec;

double elapsed =
    seconds * 1000000.0 + microseconds;

printf("Producer: Sent %d integers.\n", DATA_SIZE);
printf("Communication time: %.2f microseconds\n", elapsed);
printf("Communication efficiency: %.2f integers/second\n",
    DATA_SIZE / (elapsed / 1000000.0));
}
else {
    // Child - Consumer
    close(pipefd[1]);

    int received;
    int total = 0;

    while ((received = read(pipefd[0],
        data,
        sizeof(data))) > 0) {

        int count = received / sizeof(int);
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + | v
GNU nano 7.2 producer_consumer
    gettimeofday(&end, NULL);

    long seconds = end.tv_sec - start.tv_sec;
    long microseconds = end.tv_usec - start.tv_usec;

    double elapsed =
        seconds * 1000000.0 + microseconds;

    printf("Producer: Sent %d integers.\n", DATA_SIZE);
    printf("Communication time: %.2f microseconds\n", elapsed);
    printf("Communication efficiency: %.2f integers/second\n",
        DATA_SIZE / (elapsed / 1000000.0));
}
else {
    // Child - Consumer
    close(pipefd[1]);

    int received;
    int total = 0;

    while ((received = read(pipefd[0],
        data,
        sizeof(data))) > 0) {

        int count = received / sizeof(int);
```

```
        int count = received / sizeof(int);

        for (int i = 0; i < count; i++) {
            total += data[i];
        }

        close(pipefd[0]);

        printf("Consumer: Received data successfully.\n");
        printf("Consumer: Sum of received data = %d\n", total);
    }

    return 0;
}
```

TASK 2: Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Create first child process
    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
{
    perror("fork");
    exit(EXIT_FAILURE);
}

if (pid1 == 0)
{
    // Child 1: execute ls -l

    close(pipefd[0]);

    dup2(pipefd[1], STDOUT_FILENO);

    close(pipefd[1]);

    execlp("ls", "ls", "-l", (char *)NULL);

    perror("execlp ls");
    exit(EXIT_FAILURE);
}

// Create second child process
pid2 = fork();

if (pid2 == -1)
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2 ls_
    if (pid2 == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid2 == 0)
    {
        // Child 2: execute grep .c

        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", (char *)NULL);

        perror("execlp grep");
        exit(EXIT_FAILURE);
    }

    // Parent process
    close(pipefd[0]);
    close(pipefd[1]);
```

```
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Command executed: ls -l | grep .c\n");

    return 0;
}
```

```
Communication Efficiency: 4400110.50 integers/second
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_5$ nano ls_grep.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_5$ gcc ls_grep.c -o ls_grep
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_5$ ./ls_grep
-rw-r--r-- 1 koumudi koumudi 1263 Aug 26 17:38 ls_grep.c
-rwxr-xr-x 1 koumudi koumudi 16448 Aug 26 17:12 producer_consumer
-rw-r--r-- 1 koumudi koumudi 1840 Aug 26 17:12 producer_consumer.c
Command executed: ls -l | grep .c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_5$
```